Groth16 데이터 구조 상세 분석

목차

1. 증명 구조체 (Proof)
2. 검증 키 (VerifyingKey)
3. 증명 키 (ProvingKey)
4. 전처리된 검증 키 (PreparedVerifyingKey)
5. 메모리 레이아웃 및 최적화
6. 직렬화 및 호환성

증명 구조체 (Proof)

기본 구조

```
/// Groth16 SNARK의 증명 구조체
#[derive(Clone, Debug, PartialEq, CanonicalSerialize, CanonicalDeserialize)]
pub struct Proof<E: Pairing> {
    /// G1 그룹의 A 원소
    pub a: E::G1Affine,
    /// G2 그룹의 B 원소
    pub b: E::G2Affine,
    /// G1 그룹의 C 원소
```

```
        pub  c: E::G1Affine,
}
```

수학적 의미

각 구성 요소는 다음과 같은 수학적 의미를 가집니다:

A 원소:

```
A = α + Σ(a_i * u_i) + r * δ
```

- `α` : 구조화된 참조 문자열의 일부 (설정 단계에서 생성)
- `Σ(a_i * u_i)` : witness 값들과 QAP의 A 다항식 계수들의 선형 결합
- `r * δ` : 영지식성을 위한 무작위화 항

B 원소:

```
B = β + Σ(b_i * u_i) + s * δ
```

- `β` : 구조화된 참조 문자열의 일부
- `Σ(b_i * u_i)` : witness 값들과 QAP의 B 다항식 계수들의 선형 결합
- `s * δ` : 영지식성을 위한 무작위화 항

C 원소:

```
C = Σ(c_i * u_i) + H(τ) * t(τ) / δ + A * s + B * r - r * s * δ
```

- `Σ(c_i * u_i)` : witness와 C 다항식 계수들의 선형 결합
- `H(τ) * t(τ) / δ` : QAP 다항식 나눗셈의 몫 (핵심 계산 증명)

- `A * s + B * r - r * s * δ`: 영지식성을 위한 교차 항들

크기 및 효율성

```rust
impl<E: Pairing> Default for Proof<E> {
        fn  default() -> Self {
                Self {
                        a: E::G1Affine::default(),
                        b: E::G2Affine::default(),
                        c: E::G1Affine::default(),
                }
        }
}
```

크기 특성:

- **고정 크기**: 회로 크기와 무관하게 항상 3개 그룹 원소
- **압축 표현**: Affine 좌표계를 사용하여 메모리 효율성 극대화
- **전송 효율성**: 네트워크 전송 시 최소한의 대역폭 사용

일반적인 크기 (BLS12-381 기준):

- G1 원소: 48바이트 (압축) 또는 96바이트 (비압축)
- G2 원소: 96바이트 (압축) 또는 192바이트 (비압축)
- 전체 증명: 192바이트 (압축) 또는 384바이트 (비압축)

검증 키 (VerifyingKey)

기본 구조

```
/// Groth16 SNARK의 검증 키
#[derive(Clone, Debug, PartialEq, CanonicalSerialize, CanonicalDeserialize)]
pub struct VerifyingKey<E: Pairing> {
/// α * G (G1의 생성원)
pub alpha_g1: E::G1Affine,
/// β * H (G2의 생성원)
pub beta_g2: E::G2Affine,
/// γ * H
pub gamma_g2: E::G2Affine,
/// δ * H
pub delta_g2: E::G2Affine,
/// γ^(-1) * (β * a_i + α * b_i + c_i) * G (공개 입력용)
pub gamma_abc_g1: Vec<E::G1Affine>,
}
```

각 구성 요소의 역할

구조화된 참조 문자열 (SRS) 요소들:

1. **alpha_g1**: `α * G`
   - 페어링 방정식의 좌변에서 사용
   - A 원소의 구조적 무결성 검증
2. **beta_g2**: `β * H`

- 페어링 방정식의 우변에서 사용
- B 원소의 구조적 무결성 검증

3. **gamma_g2**: `γ * H`

- 공개 입력 처리에 사용
- 공개/비밀 입력 분리 보장

4. **delta_g2**: `δ * H`

- witness 부분 처리에 사용
- 영지식성 보장에 핵심 역할

공개 입력 처리 요소:

```
/// gamma_abc_g1의 구성
/// gamma_abc_g1[i] = γ^(-1) * (β * a_i + α * b_i + c_i) * G
```

gamma_abc_g1의 상세 분석:

- **인덱스 0**: 상수항 (공개 입력이 없을 때의 기본값)
- **인덱스 i (i≥1)**: i번째 공개 입력에 대한 계수
- **선형 결합**: 모든 공개 입력의 가중합을 효율적으로 계산

검증 키 생성 과정

```
// generator.rs에서
let  gamma_abc = cfg_iter!(a[..num_instance_variables])
.zip(&b[..num_instance_variables])
.zip(&c[..num_instance_variables])
```

```
.map(|((a, b), c)| {

// β*a_i + α*b_i + c_i 계산

let  linear_combination = beta * a + &(alpha * b) + c;

// γ^(-1)로 나누기

linear_combination * &gamma_inverse

})

.collect::<Vec<_>>();



// 그룹 원소로 변환

let  gamma_abc_g1 = g1_table.batch_mul(&gamma_abc);
```

해시 지원 (Absorb 트레이트)

```
impl<E> Absorb for VerifyingKey<E>

where

E: Pairing,

E::G1Affine: Absorb,

E::G2Affine: Absorb,

{

fn  to_sponge_bytes(&self, dest: &mut Vec<u8>) {

self.alpha_g1.to_sponge_bytes(dest);

self.beta_g2.to_sponge_bytes(dest);

self.gamma_g2.to_sponge_bytes(dest);
```

```
self.delta_g2.to_sponge_bytes(dest);

self.gamma_abc_g1

.iter()

.for_each(|g| g.to_sponge_bytes(dest));

}



fn  to_sponge_field_elements<F: PrimeField>(&self, dest: &mut Vec<F>) {

self.alpha_g1.to_sponge_field_elements(dest);

self.beta_g2.to_sponge_field_elements(dest);

self.gamma_g2.to_sponge_field_elements(dest);

self.delta_g2.to_sponge_field_elements(dest);

self.gamma_abc_g1

.iter()

.for_each(|g| g.to_sponge_field_elements(dest));

}

}
```

해시 지원의 중요성:

- **무결성 검증**: 키의 변조 여부 확인
- **커밋먼트**: 키를 해시로 커밋하여 투명성 보장
- **Fiat-Shamir**: 비대화형 프로토콜에서 챌린지 생성

증명 키 (ProvingKey)

기본 구조

```
/// Groth16 zkSNARK의 증명 키
#[derive(Clone, Debug, PartialEq, CanonicalSerialize, CanonicalDeserialize)]
pub struct ProvingKey<E: Pairing> {
/// 기본 검증 키
pub vk: VerifyingKey<E>,
/// β * G
pub beta_g1: E::G1Affine,
/// δ * G
pub delta_g1: E::G1Affine,
/// a_i * G 원소들
pub a_query: Vec<E::G1Affine>,
/// b_i * G 원소들 (G1에서)
pub b_g1_query: Vec<E::G1Affine>,
/// b_i * H 원소들 (G2에서)
pub b_g2_query: Vec<E::G2Affine>,
/// h_i * G 원소들
pub h_query: Vec<E::G1Affine>,
/// l_i * G 원소들
pub l_query: Vec<E::G1Affine>,
}
```

각 쿼리의 상세 분석

1. A 쿼리 (a_query):

```
// a_query[i] = a_i(τ) * G
// 여기서 a_i(τ)는 점 τ에서 평가된 A 다항식의 i번째 계수
```

용도: A 원소 계산 시 사용

```
// prover.rs에서
let  g_a = Self::calculate_coeff(r_g1, &pk.a_query, pk.vk.alpha_g1, &assignment)
```

2. B 쿼리 (b_g1_query, b_g2_query):

```
// b_g1_query[i] = b_i(τ) * G (G1에서)
// b_g2_query[i] = b_i(τ) * H (G2에서)
```

이중 표현의 이유:

- **G1 버전**: C 원소 계산에서 `B * r` 항 계산용
- **G2 버전**: 실제 증명의 B 원소 (페어링 검증용)

3. H 쿼리 (h_query):

```
// h_query[i] = (τ^i * Z(τ) / δ) * G
// Z(τ)는 소멸 다항식의 τ에서의 값
```

수학적 의미:

- QAP의 몫 다항식 H(x) 계산에 사용
- `H(τ) * Z(τ) / δ` 항을 효율적으로 계산

4. L 쿼리 (l_query):

```
// l_query[i] = ((β * a_i(τ) + α * b_i(τ) + c_i(τ)) / δ) * G
// witness 변수들에 대해서만 계산
```

용도: C 원소의 witness 부분 계산

```
let l_aux_acc = E::G1::msm_bigint(&pk.l_query, &aux_assignment);
```

쿼리 생성 과정

```
// generator.rs에서 쿼리 생성
let proving_key_time = start_timer!(|| "Generate the R1CS proving key");

// 배치 곱셈을 위한 전처리 테이블
let g1_table = BatchMulPreprocessing::new(g1_generator, num_scalars);
let g2_table = BatchMulPreprocessing::new(g2_generator, non_zero_b);

// A 쿼리 계산
let a_time = start_timer!(|| "Calculate A");
let a_query = g1_table.batch_mul(&a);
end_timer!(a_time);
```

```
// B 쿼리 계산 (G1)
let b_g1_time = start_timer!(|| "Calculate B G1");
let b_g1_query = g1_table.batch_mul(&b);
end_timer!(b_g1_time);

// B 쿼리 계산 (G2)
let b_g2_time = start_timer!(|| "Calculate B G2");
let b_g2_query = g2_table.batch_mul(&b);
end_timer!(b_g2_time);

// H 쿼리 계산
let h_time = start_timer!(|| "Calculate H");
let h_scalars = QAP::h_query_scalars::<_, D<E::ScalarField>>(
m_raw - 1, t, zt, delta_inverse
)?;
let h_query = g1_table.batch_mul(&h_scalars);
end_timer!(h_time);

// L 쿼리 계산
let l_time = start_timer!(|| "Calculate L");
let l_query = g1_table.batch_mul(&l);
```

```
end_timer!(l_time);
```

메모리 사용량 분석

증명 키의 크기 (변수 n개, 제약 조건 m개 기준):

- `vk` : 검증 키 크기 (공개 입력 수에 비례)
- `a_query` : n개 G1 원소
- `b_g1_query` : n개 G1 원소
- `b_g2_query` : n개 G2 원소
- `h_query` : m개 G1 원소
- `l_query` : (n - 공개입력수)개 G1 원소

총 메모리 사용량 (BLS12-381 기준):

```
크기 ≈ (3n + m) * 48바이트 + n * 96바이트
≈ (4.5n + m) * 48바이트
```

대규모 회로의 경우 수 GB에 달할 수 있습니다.

전처리된 검증 키 (PreparedVerifyingKey)

기본 구조

```
/// 빠른 검증을 위해 전처리된 검증 키
#[derive(Clone, Debug, PartialEq, CanonicalSerialize, CanonicalDeserialize)]
pub struct PreparedVerifyingKey<E: Pairing> {
```

```
        /// 원본 검증 키
        pub vk: VerifyingKey<E>,
        /// 미리 계산된 e(α * G, β * H)
        pub alpha_g1_beta_g2: E::TargetField,
        /// 전처리된 -γ * H (페어링용)
        pub gamma_g2_neg_pc: E::G2Prepared,
        /// 전처리된 -δ * H (페어링용)
        pub delta_g2_neg_pc: E::G2Prepared,
}
```

전처리의 목적과 효과

1. 페어링 값 미리 계산:

```
// 매번 계산하는 대신 미리 저장
alpha_g1_beta_g2: E::pairing(vk.alpha_g1, vk.beta_g2).0
```

성능 향상:

- 검증 시마다 페어링 계산 생략
- 약 25% 검증 시간 단축

2. G2 원소 전처리:

```
// 페어링에 최적화된 형태로 변환
gamma_g2_neg_pc: vk.gamma_g2.into_group().neg().into_affine().into()
```

```
delta_g2_neg_pc: vk.delta_g2.into_group().neg().into_affine().into()
```

최적화 효과:

- Miller 루프에서 사용할 수 있는 형태로 미리 변환
- 페어링 계산 시 전처리 오버헤드 제거

전처리 과정

```
/// 검증 키를 검증에 최적화된 형태로 전처리
pub fn prepare_verifying_key<E: Pairing>(vk: &VerifyingKey<E>) -> PreparedVeri
PreparedVerifyingKey {
vk: vk.clone(),
// e(α*G, β*H) 미리 계산
alpha_g1_beta_g2: E::pairing(vk.alpha_g1, vk.beta_g2).0,
// -γ*H를 페어링에 최적화된 형태로 전처리
gamma_g2_neg_pc: vk.gamma_g2.into_group().neg().into_affine().into(),
// -δ*H를 페어링에 최적화된 형태로 전처리
delta_g2_neg_pc: vk.delta_g2.into_group().neg().into_affine().into(),
}
}
```

변환 트레이트 구현

```
impl<E: Pairing> From<PreparedVerifyingKey<E>> for VerifyingKey<E> {
```

```
fn from(other: PreparedVerifyingKey<E>) -> Self {
other.vk
}
}

impl<E: Pairing> From<VerifyingKey<E>> for PreparedVerifyingKey<E> {
fn from(other: VerifyingKey<E>) -> Self {
crate::prepare_verifying_key(&other)
}
}
```

편의성 제공:

- 자동 변환을 통한 사용성 향상
- 기존 코드와의 호환성 보장

메모리 레이아웃 및 최적화

Affine vs Projective 좌표계

```
// 저장 시에는 Affine 사용 (메모리 효율적)
pub a: E::G1Affine,
pub b: E::G2Affine,
pub c: E::G1Affine,
```

```
// 계산 시에는 Projective 사용 (연산 효율적)
let mut g_c = s_g_a.into_group(); // Projective로 변환
g_c += &r_g1_b; // 효율적인 덧셈
g_c -= &r_s_delta_g1;
let c = g_c.into_affine(); // 다시 Affine으로 변환
```

좌표계 선택 기준:

- **Affine**: 메모리 효율적, 직렬화에 적합
- **Projective**: 연산 효율적, 역원 계산 불필요

메모리 정렬 최적화

```
#[repr(C)] // C 스타일 메모리 레이아웃 (필요시)
pub struct Proof<E: Pairing> {
    pub a: E::G1Affine, // 캐시 라인 정렬 고려
    pub b: E::G2Affine,
    pub c: E::G1Affine,
}
```

배치 처리 최적화

```
// 여러 증명을 동시에 처리할 때
pub fn batch_verify(
    pvk: &PreparedVerifyingKey<E>,
```

```
        proofs: &[Proof<E>],
        public_inputs: &[Vec<E::ScalarField>],
) -> Result<bool, SynthesisError> {
// 배치 페어링을 사용하여 효율성 향상
// 개별 검증보다 약 30% 빠름
}
```

직렬화 및 호환성

표준 직렬화

```
#[derive(CanonicalSerialize, CanonicalDeserialize)]
pub struct Proof<E: Pairing> {
        pub a: E::G1Affine,
        pub b: E::G2Affine,
        pub c: E::G1Affine,
}
```

직렬화 형식:

- **압축 모드**: 최소 크기, 약간의 계산 오버헤드
- **비압축 모드**: 빠른 역직렬화, 더 큰 크기

네트워크 전송 최적화

```
// 압축된 형태로 전송
```

```rust
let compressed_proof = proof.serialize_compressed()?;


// 수신 측에서 압축 해제
let proof = Proof::deserialize_compressed(&compressed_proof[..])?;
```

버전 호환성

```rust
// 버전 정보 포함
#[derive(CanonicalSerialize, CanonicalDeserialize)]
pub struct VersionedProof<E: Pairing> {
    pub version: u32,
    pub proof: Proof<E>,
}
```

호환성 보장:

- 이전 버전과의 호환성 유지
- 점진적 업그레이드 지원
- 명확한 버전 관리

크로스 플랫폼 지원

```rust
// 엔디안 독립적 직렬화
impl<E: Pairing> Proof<E> {
    pub fn to_bytes(&self) -> Vec<u8> {
```

```
        // 빅엔디안으로 통일하여 플랫폼 독립성 보장
        }
        pub fn from_bytes(bytes: &[u8]) -> Result<Self, Error> {
        // 엔디안 변환 처리
        }
}
```

이러한 데이터 구조들은 Groth16의 효율성과 보안성을 동시에 보장하면서, 실제 사용에서 필요한 모든 기능을 제공합니다. 각 구조체는 수학적 정확성과 구현 효율성 사이의 균형을 잘 맞추고 있으며, 확장성과 호환성도 고려하여 설계되었습니다.